

SQL – Data Manipulation Language

Carla Teixeira Lopes

Bases de Dados

Licenciatura em Engenharia Informática e Computação, FEUP+FCUP

Based on Jennifer Widom slides

Agenda

Introduction

The JOIN family of operators

~~Basic SELECT Statement~~

Aggregation

~~Table Variables and Set Operators~~

Null values

~~Subqueries in WHERE clauses~~

Data Modification statements

Subqueries in FROM and SELECT clauses

Subqueries in FROM and SELECT

SELECT $A_1, A_2, \dots, A_n$ 
FROM $R_1, R_2, \dots, R_m$  Expressions involving subqueries
WHERE condition

Subqueries are nested SELECT statements

Subqueries in FROM generate a table to be used in the query

Subqueries in SELECT produce a value that comes out of the query

Subqueries in the FROM clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT  sID, sName, GPA, GPA*(HS/1000) as scaledGPA
FROM    Student
WHERE   GPA*(HS/1000)-GPA>1.0 OR GPA - GPA*(HS/1000) > 1.0;
```

Return all students whose scaledGPA changes GPA by more than 1

Can we simplify this query?

sID	sName	GPA	scaledGPA
234	Bob	3.6	5.4
345	Craig	3.5	1.75
567	Edward	2.9	5.8
678	Fay	3.8	0.76
876	Irene	3.9	1.56
765	Jay	2.9	4.35
543	Craig	3.4	6.8

Subqueries in the FROM clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName, GPA, GPA*(HS/1000) as scaledGPA
FROM Student
WHERE GPA*(HS/1000)-GPA>1.0 OR GPA – GPA*(HS/1000) > 1.0;
```



```
SELECT sID, sName, GPA, GPA*(HS/1000) as scaledGPA
FROM Student
WHERE abs(GPA*(HS/1000)-GPA)>1.0;
```

<u>sID</u>	sName	GPA	scaledGPA
234	Bob	3.6	5.4
345	Craig	3.5	1.75
567	Edward	2.9	5.8
678	Fay	3.8	0.76
876	Irene	3.9	1.56
765	Jay	2.9	4.35
543	Craig	3.4	6.8

Subqueries in the FROM clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName, GPA, GPA*(HS/1000) as scaledGPA
FROM Student
WHERE abs(GPA*(HS/1000)-GPA) > 1.0;
```

sID	sName	GPA	scaledGPA
234	Bob	3.6	5.4
345	Craig	3.5	1.75
567	Edward	2.9	5.8
678	Fay	3.8	0.76
876	Irene	3.9	1.56
765	Jay	2.9	4.35
543	Craig	3.4	6.8

↓

```
SELECT *
FROM (select sID, sName, GPA, GPA*(HS/1000) as scaledGPA
      from Student) G
WHERE abs(scaledGPA - GPA) > 1.0;
```

Subqueries in the SELECT clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT DISTINCT College.cName, state, GPA
FROM   College, Apply A1, Student S1
WHERE  College.cName = A1.cName
      AND A1.sID = S1.sID
      AND GPA >= all
      (select GPA from Student S2, Apply A2
       where S2.sID = A2.sID
       and A2.cName = College.cName);
```

<u>cName</u>	state	GPA
Stanford	CA	3.9
Berkeley	CA	3.9
Cornell	NY	3.9
MIT	MA	3.9

How to rewrite it using a
subquery in SELECT?

Return colleges paired with the highest GPA of their applicants

Subqueries in the SELECT clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName, state,  
(SELECT DISTINCT GPA  
FROM   Apply A1, Student S1  
WHERE  College.cName = A1.cName  
        AND A1.sID = S1.sID  
        AND GPA >= all  
        (select GPA from Student S2, Apply A2  
         where S2.sID = A2.sID  
         and A2.cName = College.cName)) AS GPA  
FROM College;
```

Computes the
highest GPA for
the college

Subqueries in the SELECT clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Query that returns colleges paired with the name of the applicants

```
SELECT cName, state,  
(SELECT DISTINCT sName  
FROM   Apply, Student  
WHERE  College.cName = Apply.cName  
       AND Apply.sID = Student.sID) AS sName  
FROM College;
```



Error. Why?



subquery returns
more than 1 row



subquery in SELECT can only
return 1 column of 1 tuple

Agenda

Introduction

The JOIN family of operators

Basic SELECT Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

The JOIN family of operators

SELECT $A_1, A_2, \dots, A_n$
FROM $R_1, R_2, \dots, R_m$
WHERE *condition*



Implicit join

Explicit Joins

Inner join on *condition*

$\bowtie_{\theta}$

Natural join

$\bowtie$

Inner join using (*attrs*)

$\bowtie$ explicitly listing the attributes to be equated

Left | Right | Full Outer Join

Combines tuples as in $\bowtie_{\theta}$ but when they don't match, they are added to the result with NULL values

None of these operators adds expressive power to SQL

Inner join on *condition*

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT DISTINCT sName, major
FROM   Student, Apply
WHERE  Student.sID = Apply.sID;
```

```
SELECT DISTINCT sName, major
FROM   Student INNER JOIN Apply
ON     Student.sID = Apply.sID;
```

```
SELECT DISTINCT sName, major
FROM   Student JOIN Apply
ON     Student.sID = Apply.sID;
```

Equivalent to:

INNER JOIN is the DEFAULT
JOIN operator in SQL

sName	major
Amy	CS
Amy	EE
Bob	biology
Craig	bioengineering
Craig	CS
Craig	EE
Fay	history
Helen	CS
Irene	CS
Irene	biology
Irene	marine biology
Jay	history
Jay	psychology

Inner join on *condition*: example 2

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sName, GPA
FROM   Student, Apply
WHERE  Student.sID=Apply.sID AND HS<1000 AND major='CS' AND cName='Stanford';
```

↓ Equivalent to:

```
SELECT sName, GPA
FROM   Student JOIN Apply
ON     Student.sID=Apply.sID
WHERE  HS<1000 AND major='CS' AND cName='Stanford';
```

↓ Equivalent to:

```
SELECT sName, GPA
FROM   Student JOIN Apply
ON     Student.sID=Apply.sID AND HS<1000 AND major='CS' AND cName='Stanford';
```

sName	GPA
Helen	3.7
Irene	3.9

Inner join on *condition* with 3 relations

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT Apply.sID, sName, GPA, Apply.cName, enr
FROM   Apply, Student, College
WHERE  Apply.sID = Student.sID AND Apply.cName = College.cName;
```

```
SELECT Apply.sID, sName, GPA, Apply.cName, enr
FROM   Apply JOIN Student JOIN College
ON      Apply.sID = Student.sID AND Apply.cName = College.cName;
```

Runs on SQLite but not in every system

```
SELECT Apply.sID, sName, GPA, Apply.cName, enr
FROM   (Apply JOIN Student ON Apply.sID = Student.sID) JOIN College
ON      Apply.cName = College.cName;
```

Some systems (e.g.: Postgres) requires the join operators to be binary

Interaction between query and processor

SQL systems tend to follow the structure that is provided by the JOIN operators and parenthesis

```
SELECT Apply.sID, sName, GPA, Apply.cName, enr
FROM   (Apply JOIN Student ON Apply.sID = Student.sID) JOIN College
ON     Apply.cName = College.cName;
```



Typically, Apply will be joined with Student first

The order by things are done affects query performance

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Natural join

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

It joins tables based on the attributes with the same name and keeps only one of those attributes

```
SELECT DISTINCT sName, major
FROM Student, Apply
WHERE Student.sID = Apply.sID;
```



Equivalent to:

```
SELECT DISTINCT sName, major
FROM Student NATURAL JOIN Apply;
```

sName	major
Amy	CS
Amy	EE
Bob	biology
Craig	bioengineering
Craig	CS
Craig	EE
Fay	history
Helen	CS
Irene	CS
Irene	biology
Irene	marine biology
Jay	history
Jay	psychology

Natural join with additional conditions

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sName, GPA
FROM   Student JOIN Apply
ON Student.sID=Apply.sID
WHERE HS<1000 AND major='CS' AND cName='Stanford';
```

Equivalent to:

```
SELECT sName, GPA
FROM   Student NATURAL JOIN Apply
WHERE HS<1000 AND major='CS' AND cName='Stanford';
```

sName	GPA
Helen	3.7
Irene	3.9

Inner join using (*attrs*)

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Using lists the attributes that should be acquainted across the 2 relations

Only attributes that appear in both relations

```
SELECT sName, GPA
FROM   Student NATURAL JOIN Apply
WHERE  HS<1000 AND major='CS' AND cName='Stanford';
```

sName	GPA
Helen	3.7
Irene	3.9

↓ Equivalent to:

```
SELECT sName, GPA
FROM   Student JOIN Apply using(sID)
WHERE  HS<1000 AND major='CS' AND cName='Stanford';
```

Better practice than using the natural join

Join with more than 1 instance of a relation

```
SELECT S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA
FROM   Student S1, Student S2
WHERE  S1.GPA=S2.GPA AND S1.sID < S2.sID;
```

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Finds pairs of students with the same GPA

```
SELECT S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA
FROM   Student S1 join Student S2 using(GPA)
ON      S1.sID < S2.sID;
```

————→ Error
 ↓

Using and ON cannot
be used in combination

Join with more than 1 instance of a relation

```
SELECT S1.sID, S1.sName, S1.GPA, S2.sID, S2.sName, S2.GPA
FROM   Student S1 join Student S2 using(GPA)
WHERE  S1.sID < S2.sID;
```

S1.sID	S1.sName	S1.GPA	S2.sID	S2.sName	S2.GPA
123	Amy	3.9	456	Doris	3.9
123	Amy	3.9	876	Irene	3.9
123	Amy	3.9	654	Amy	3.9
456	Doris	3.9	876	Irene	3.9
456	Doris	3.9	654	Amy	3.9
567	Edward	2.9	765	Jay	2.9
654	Amy	3.9	876	Irene	3.9
543	Craig	3.4	789	Gary	3.4

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Left outer join

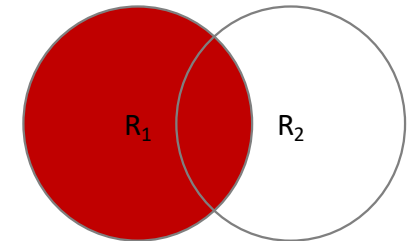
College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Takes any tuples on the left side and if they don't have a match on a tuple from the right, they are still added to the result and padded with NULL values

Tuples with no matches are *dangling tuples*



```
SELECT sName, sID, cName, major  
FROM Student LEFT OUTER JOIN Apply using(sID);
```

LEFT JOIN



abbreviation

Left outer join

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

SELECT sName, sID, cName, major

FROM Student INNER JOIN Apply using(sID);

sName	sID	cName	major
Amy	123	Cornell	EE
Amy	123	Berkeley	CS
Amy	123	Stanford	EE
Amy	123	Stanford	CS
Bob	234	Berkeley	biology
Craig	345	Cornell	EE
Craig	345	Cornell	CS
Craig	345	Cornell	bioengineering
Craig	345	MIT	bioengineering
Fay	678	Stanford	history
...	...	...	...

Students who have applied somewhere

SELECT sName, sID, cName, major

FROM Student LEFT JOIN Apply using(sID);

sName	sID	cName	major
Amy	123	Cornell	EE
Amy	123	Berkeley	CS
Amy	123	Stanford	EE
Amy	123	Stanford	CS
Bob	234	Berkeley	biology
Craig	345	Cornell	EE
Craig	345	Cornell	CS
Craig	345	Cornell	bioengineering
Craig	345	MIT	bioengineering
Doris	456	NULL	NULL
...	...	...	...

Students who have applied somewhere plus students who haven't yet applied anywhere

Natural left outer join

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sName, sID, cName, major
FROM Student NATURAL LEFT JOIN Apply;
```

sName	sID	cName	major
Amy	123	Cornell	EE
Amy	123	Berkeley	CS
Amy	123	Stanford	EE
Amy	123	Stanford	CS
Bob	234	Berkeley	biology
Craig	345	Cornell	EE
Craig	345	Cornell	CS
Craig	345	Cornell	bioengineering
Craig	345	MIT	bioengineering
Doris	456	NULL	NULL
...	...	...	...

Left outer join

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

How to rewrite the left outer join without using it?

```
SELECT sName, Student.sID, cName, major
FROM Student, Apply
Where Student.sID=Apply.sID
UNION
SELECT sName, sID, NULL, NULL
FROM Student
WHERE sID NOT IN (select sID from Apply)
```

sName	sID	cName	major
Amy	123	Cornell	EE
Amy	123	Berkeley	CS
Amy	123	Stanford	EE
Amy	123	Stanford	CS
Bob	234	Berkeley	biology
Craig	345	Cornell	EE
Craig	345	Cornell	CS
Craig	345	Cornell	bioengineering
Craig	345	MIT	bioengineering
Doris	456	NULL	NULL
...	...	...	...

Right outer join

College(cName, state, enr)

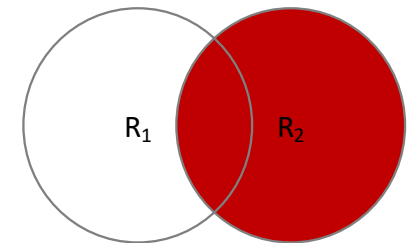
Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Takes any tuples on the right side and if they don't have a match on a tuple from the left, they are still added to the result and padded with NULL values

We can also use the left outer join for the same effect swapping the order of the relations

```
SELECT sName, sID, cName, major  
FROM Student RIGHT OUTER JOIN Apply using(sID);
```



Full outer join

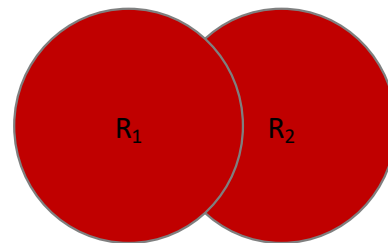
College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

To include unmatched tuples from both sides of a join

```
SELECT sName, sID, cName, major  
FROM Student FULL OUTER JOIN Apply using(sID);
```



Full outer join

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

How can we express a full outer join without using it?

```
SELECT sName, Student.sID, cName, major  
FROM Student LEFT JOIN Apply using(sID)
```

```
UNION
```

```
SELECT sName, Student.sID, cName, major  
FROM Student RIGHT JOIN Apply using(sID);
```

→ Automatically
eliminates duplicates

Full outer join

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

How can we express a full outer join without using joins?

```
SELECT sName, Student.sID, cName, major
```

```
FROM Student, Apply
```

```
Where Student.sID=Apply.sID
```

```
UNION
```

```
SELECT sName, sID, NULL, NULL
```

```
FROM Student
```

```
WHERE sID NOT IN (select sID from Apply)
```

```
UNION
```

```
SELECT NULL, sID, cName, major
```

```
FROM Apply
```

```
WHERE sID NOT IN (select sID from Student)
```

Outer joins and commutativity

Commutativity $(A \text{ op } B) = (B \text{ op } A)$

Left and Right Outer Joins are not commutative

Full Outer Join is commutative

Outer joins and associativity

Associativity $(A \text{ op } B) \text{ op } C = A \text{ op } (B \text{ op } C)$

T1		T2		T3	
A	B	B	C	A	C
1	2	2	3	4	5

SELECT A, B, C

FROM (T1 natural full outer join T2) natural full outer join T3;

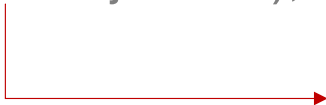
SELECT A, B, C

FROM T1 natural full outer join (T2 natural full outer join T3);

Left and right outer joins are not associative either



A	B	C
1	2	3
4	NULL	5



A	B	C
4	NULL	5
NULL	2	3
1	2	NULL

Outer joins summary

Left outer join

Include the left tuple even if there's no match

Right outer join

Include the right tuple even if there's no match

Full outer join

Include the both left and right tuples even if there's no match

Agenda

Introduction

The JOIN family of operators

Basic SELECT Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

Aggregation

Perform aggregation over sets of values in multiple rows

Basic functions

min, max, sum, avg, count

Except *count*, all aggregations apply to a single attribute

SELECT $A_1, A_2, \dots, A_n$

FROM $R_1, R_2, \dots, R_m$

WHERE condition

GROUP BY columns

HAVING condition



New clauses

A first aggregation query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *  
FROM Student;
```

sID	sName	GPA	HS
123	Amy	3.9	1000
234	Bob	3.6	1500
345	Craig	3.5	500
456	Doris	3.9	1000
567	Edward	2.9	2000
678	Fay	3.8	200
789	Gary	3.4	800
987	Helen	3.7	800
876	Irene	3.9	400
765	Jay	2.9	1500
654	Amy	3.9	1000
543	Craig	3.4	2000

```
SELECT avg(GPA)  
FROM Student;
```

avg(GPA)
3.566

A second aggregation query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *  
FROM Student, Apply  
WHERE Student.sID = Apply.sID  
and major='CS';
```



sID	sName	GPA	HS	sID1	cName	major	dec
123	Amy	3.9	1000	123	Stanford	CS	Y
123	Amy	3.9	1000	123	Berkeley	CS	Y
345	Craig	3.5	500	345	Cornell	CS	Y
987	Helen	3.7	800	987	Stanford	CS	Y
987	Helen	3.7	800	987	Berkeley	CS	Y
876	Irene	3.9	400	876	Stanford	CS	N
543	Craig	3.4	2000	543	MIT	CS	N

```
SELECT min(GPA)  
FROM Student, Apply  
WHERE Student.sID = Apply.sID  
and major='CS';
```



min(GPA)
3.4

Lowest GPA of students
applying to CS

A third aggregation query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *  
FROM Student, Apply  
WHERE Student.sID = Apply.sID  
and major='CS';
```



sID	sName	GPA	HS	sID1	cName	major	dec
123	Amy	3.9	1000	123	Stanford	CS	Y
123	Amy	3.9	1000	123	Berkeley	CS	Y
345	Craig	3.5	500	345	Cornell	CS	Y
987	Helen	3.7	800	987	Stanford	CS	Y
987	Helen	3.7	800	987	Berkeley	CS	Y
876	Irene	3.9	400	876	Stanford	CS	N
543	Craig	3.4	2000	543	MIT	CS	N

```
SELECT avg(GPA)  
FROM Student, Apply  
WHERE Student.sID = Apply.sID  
and major='CS';
```



avg(GPA)
3.71

Average GPA of students
applying to CS?

A third aggregation query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *  
FROM Student  
WHERE sID in (select sID from  
Apply where major='CS');
```

sID	sName	GPA	HS
123	Amy	3.9	1000
345	Craig	3.5	500
987	Helen	3.7	800
876	Irene	3.9	400
543	Craig	3.4	2000

```
SELECT avg(GPA)  
FROM Student  
WHERE sID in (select sID from  
Apply where major='CS');
```

avg(GPA)
3.68

Average GPA of students
applying to CS

A first count query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *  
FROM   College  
WHERE  enr > 15000;
```

cName	state	enr
Berkeley	CA	36000
Cornell	NY	21000

```
SELECT count(*)  
FROM   College  
WHERE  enr > 15000;
```

count(*)
2

Number of colleges
bigger than 15000

A second count query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT count(sID)
FROM   Apply
WHERE  cName = 'Cornell';
```

count(*)

6

Number of students
applying to Cornell?



Number of applications
to Cornell



COUNT applies to duplicates,
unless otherwise stated

```
SELECT count(distinct sID)
FROM   Apply
WHERE  cName = 'Cornell';
```

count(distinct sID)

3

Number of students
applying to Cornell

A third count query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *
FROM   Student S1
WHERE  (select count(*) from Student S2
        where S2.sID <> S1.sID and S2.GPA = S1.GPA) =
        (select count(*) from Student S2
        where S2.sID <> S1.sID and S2.sizeHS = S1.sizeHS);
```

What does it compute?

Students such that number of other students with same GPA is equal to number of other students with same HS size

A fourth count query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT CS.avgGPA – NonCS.avgGPA
FROM   (select avg(GPA) as avgGPA from Student
        where sID in (
            select sID from Apply where major = 'CS')) as CS,
        (select avg(GPA) as avgGPA from Student
        where sID not in (
            select sID from Apply where major = 'CS')) as NonCS;
```

**CS.avgGPA-
NonCS.avgGPA**

0.19

What does it compute?

Amount by which the average GPA of students applying to CS exceeds average of students not applying to CS

A fourth count query

Compute the “amount by which the average GPA of students applying to CS exceeds average of students not applying to CS” using subqueries in the select clause

```
SELECT (select avg(GPA) from Student
        where sID in (
            select sID from Apply where major = 'CS')) -
(select avg(GPA) from Student
        where sID not in (
            select sID from Apply where major = 'CS')) as d
FROM Student;
```

d
0.19
0.19
0.19
0.19
...



Duplicates: difference
is computed for each
student

A fourth count query

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Compute the “amount by which the average GPA of students applying to CS exceeds average of students not applying to CS” using subqueries in the select clause

```
SELECT DISTINCT (select avg(GPA) as avgGPA from Student
    where sID in (
        select sID from Apply where major = 'CS')) -
    (select avg(GPA) as avgGPA from Student
    where sID not in (
        select sID from Apply where major = 'CS')) as d
FROM Student;
```

d
0.19

Another example

```
SELECT SUM(price * quantity)
FROM Purchase
WHERE product = 'cake'
```



Sum(price*quantity)
50

(= 1*20 + 1.50*20)

Purchase

Product	Date	Price	Quantity
cake	21/10	1	20
banana	3/10	0.5	10
banana	10/10	1	10
cake	25/10	1.50	20

Group by clause

Only used in conjunction with aggregation

It partitions a relation by values of a given attribute or set of attributes

Semantics of the query

1. Compute the FROM and WHERE clauses
2. Group by the attributes in the GROUP BY
3. Compute the SELECT clause: grouped attributes and aggregates

SELECT	S
FROM	$R_1, \dots, R_n$
WHERE	C_1
GROUP BY	$a_1, \dots, a_k$

Group by clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *  
FROM Apply  
ORDER BY cName;
```

```
SELECT cName, count(*)  
FROM Apply  
GROUP BY cName;
```

1. FROM and WHERE

sID	cName	major	dec
123	Berkeley	CS	Y
234	Berkeley	biology	N
987	Berkeley	CS	Y

2. Group By

sID	cName	major	dec
123	Cornell	EE	Y
345	Cornell	bio	N
...	...	...	...

sID	cName	major	dec
123	Berkeley	CS	Y
234		biology	N
987		CS	Y
123	Cornell	EE	Y
345		bioengineering	N
...		...	...

3. Select

cName	count(*)
Berkeley	3
Cornell	6
MIT	4
Stanford	6

A second query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT *  
FROM College  
ORDER BY state;
```

<u>cName</u>	state	enr
Stanford	CA	15000
Berkeley	CA	36000
MIT	MA	10000
Cornell	NY	21000

```
SELECT state, sum(enr)  
FROM College  
GROUP BY state;
```

state	sum(enr)
CA	51000
MA	10000
NY	21000

College enrollments by state

A third query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName, major, GPA
FROM Student, Apply
WHERE Student.sID = Apply.sID
ORDER BY cName, major;
```

cName	major	GPA
Berkeley	biology	3.6
Berkeley	CS	3.9
Berkeley	CS	3.7
Cornell	bioengineering	3.5
...	...	...

```
SELECT cName, major, min(GPA),
max(GPA)
FROM Student, Apply
WHERE Student.sID = Apply.sID
GROUP BY cName, major;
```

cName	major	min(GPA)	max(GPA)
Berkeley	biology	3.6	3.6
Berkeley	CS	3.7	3.9
Cornell	bioengineering	3.5	3.5
...	...	...	...

Minimum and maximum GPAs of applicants to each college and major

A fourth query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Find the largest spread of GPAs in colleges and majors

Step 1: Find the spread of GPAs of applicants for each college and major

```
SELECT mx-mn
```

```
FROM ( SELECT cName, major, min(GPA) as mn, max(GPA) as mx
```

```
      FROM Student, Apply
```

```
      WHERE Student.sID = Apply.sID
```

```
      GROUP BY cName, major ) M;
```

mx-mn
0
0.19
0
...

A fourth query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Find the largest spread of GPAs in colleges and majors

Step 2: Find the largest spread

```
SELECT max(mx-mn)
FROM ( SELECT cName, major, min(GPA) as mn, max(GPA) as mx
      FROM Student, Apply
      WHERE Student.sID = Apply.sID
      GROUP BY cName, major ) M;
```

max(mx-mn)

0.899

A fifth query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT Student.sID, cName
FROM Student, Apply
WHERE Student.sID = Apply.sID
ORDER BY Student.sID;
```

sID	cName
123	Stanford
123	Stanford
123	Berkeley
123	Cornell
234	Berkeley
...	...

```
SELECT Student.sID, count(distinct cName)
FROM Student, Apply
WHERE Student.sID = Apply.sID
GROUP BY Student.sID;
```

sID	Count(distinct cName)
123	3
234	1
345	2
...	...

Number of colleges applied to by each student

A fifth query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

What if we also want the student's name?

```
SELECT Student.sID, sName, count(distinct cName)
FROM Student, Apply
WHERE Student.sID = Apply.sID
GROUP BY Student.sID;
```

It only worked because, for each
sID, we only have one sName



sID	sName	count(distinct cName)
123	Amy	3
234	Bob	1
345	Craig	2
...		...

A fifth query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT Student.sID, sName, count(distinct cName), cName
FROM Student, Apply
WHERE Student.sID = Apply.sID
GROUP BY Student.sID;
```

sID	sName	count(distinct cName)	cName
123	Amy	3	Stanford
234	Bob	1	Berkeley
345	Craig	2	MIT
...		...	...

→ It chooses a random value
from the group to include

↓
Some systems throw an
error in this situation

A fifth query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT Student.sID, count(distinct cName)
FROM Student, Apply
WHERE Student.sID = Apply.sID
GROUP BY Student.sID;
```

sID	Count(distinct cName)
123	3
234	1
345	2
...	...



What if want to list students who haven't applied anywhere with a 0 in the count?

A fifth query with group by

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT Student.sID, count(distinct cName)
FROM Student, Apply
WHERE Student.sID = Apply.sID
GROUP BY Student.sID
UNION
SELECT sID, 0
FROM Student
WHERE sID not in (select sID from Apply);
```

sID	Count(distinct cName)
876	2
987	2
456	0
567	0
...	...

Having clause

Applies conditions to the results of aggregate functions

Check conditions that involve the entire group

Where clause applies to one tuple at a time

Applied after the GROUP BY clause

SELECT	S	→	attributes $a_1, \dots, a_k$ and/or aggregates over other attributes
FROM	$R_1, \dots, R_n$		
WHERE	C_1	→	any condition on the attributes in $R_1, \dots, R_n$
GROUP BY	$a_1, \dots, a_k$		
HAVING	C_2	→	any condition on the aggregate expressions

Having clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

List the colleges with fewer than 5 applications

SELECT cName

FROM Apply

GROUP BY cName

HAVING count(*) < 5;

cName
Berkeley
MIT

Having clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

What if we're interested in the colleges that have fewer than 5 applicants

```
SELECT cName
```

```
FROM Apply
```

```
GROUP BY cName
```

```
HAVING count(distinct sID) < 5;
```

cName
Berkeley
Cornell
MIT

A final having clause

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

SELECT major

FROM Student, Apply

WHERE Student.sID = Apply.sID

GROUP BY major

HAVING max(GPA) < (SELECT avg(GPA) FROM Student);

major

bioengineering

psychology

What does it compute?

Majors whose applicant's maximum GPA is below the average

Group-by versus subqueries

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName, count(*) AS cnt
FROM Apply
GROUP BY cName;
```

```
SELECT DISTINCT cName,
    (SELECT count(*)
     FROM Apply A2
     WHERE A2.cName = A1.cName) AS cnt
FROM Apply A1;
```

cName	cnt
Berkeley	3
Cornell	6
MIT	4
Stanford	6

Group-by versus subqueries

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName
FROM Apply
GROUP BY cName
HAVING count(*) < 5;
```



No need for DISTINCT:
automatically from GROUP BY

cName
Berkeley
MIT

```
SELECT DISTINCT cName
FROM Apply A1
WHERE 5 > (SELECT count(*) FROM Apply A2 WHERE A2.cName = A1.cName);
```

Every *group by* and *having* query can be written without using those clauses

Group-by versus subqueries

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT cName
FROM Apply
GROUP BY cName
HAVING count(*) < 5;
```

This is SQL by an expert

```
SELECT DISTINCT cName
FROM Apply A1
WHERE 5 > (SELECT count(*)
           FROM Apply A2
           WHERE A2.cName = A1.cName);
```

This is SQL by a novice

Which way is more efficient?

How many times do we do a SFW query in each case?

With GROUP BY can be much more efficient!

Aggregation summary

SELECT	S	→	attributes $a_1, \dots, a_k$ and/or aggregates over other attributes
FROM	$R_1, \dots, R_n$		
WHERE	C_1	→	any condition on the attributes in $R_1, \dots, R_n$
GROUP BY	$a_1, \dots, a_k$		
HAVING	C_2	→	any condition on the aggregate expressions

Evaluation steps

1. Evaluate FROM-WHERE: apply condition C_1 on the attributes in $R_1, \dots, R_n$
2. GROUP BY the attributes $a_1, \dots, a_k$
3. Apply condition C_2 to each group
4. Compute aggregates in S and return the result

Agenda

Introduction

The JOIN family of operators

Basic SELECT Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

NULL values

Unless specified otherwise, any value in an attribute can take on the special value NULL

NULL usually means that the value is undefined or unknown or not applicable

We will see what happens when we have NULL values and we run queries over the database

NULL values for numerical operations

NULL -> NULL

If $x = \text{NULL}$ then

$4 \cdot (3 - x) / 7$ is NULL

NULL values for Boolean operations

Conditions are evaluated using a three-value logic: TRUE, FALSE, UNKNOWN

If $x = \text{NULL}$ then $x = \text{'Joe'}$ is UNKNOWN

Considering TRUE is 1.0, UNKNOWN is 0.5 and FALSE is 0.0

$C1 \text{ AND } C2 = \min(C1, C2)$

$C1 \text{ OR } C2 = \max(C1, C2)$

$\text{NOT } C1 = 1 - C1$

Rule in SQL: include only tuples that yield TRUE (1.0)

A first query with NULL values

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName, GPA
FROM Student
WHERE GPA > 3.5;
```



sID	sName	GPA
123	Amy	3.9
234	Bob	3.6
456	Doris	3.9
678	Fay	3.8
987	Helen	3.7
876	Irene	3.9
654	Amy	3.9

Student

sID	sName	GPA	HS
123	Amy	3.9	1000
234	Bob	3.6	1500
345	Craig	3.5	500
456	Doris	3.9	1000
567	Edward	2.9	2000
678	Fay	3.8	200
789	Gary	3.4	800
987	Helen	3.7	800
876	Irene	3.9	400
765	Jay	2.9	1500
654	Amy	3.9	1000
543	Craig	3.4	2000
432	Kevin	NULL	1500
321	Lori	NULL	2500

2 additional students

A second query with NULL values

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName, GPA
FROM Student
WHERE GPA <= 3.5;
```

<u>sID</u>	sName	GPA
345	Craig	3.5
567	Edward	2.9
789	Gary	3.4
765	Jay	2.9
543	Craig	3.4

A third query with NULL values

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName, GPA
FROM Student
WHERE GPA > 3.5 OR GPA <= 3.5;
```

↓
Tautology

<u>sID</u>	sName	GPA
123	Amy	3.9
234	Bob	3.6
345	Craig	3.5
456	Doris	3.9
567	Edward	2.9
678	Fay	3.8
789	Gary	3.4
987	Helen	3.7
876	Irene	3.9
765	Jay	2.9
654	Amy	3.9
543	Craig	3.4

Even with a tautology, we might not get all the data

A fourth query with NULL values

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

To make this query return all the students

```
SELECT sID, sName, GPA
```

```
FROM Student
```

```
WHERE GPA > 3.5 OR GPA <= 3.5 OR GPA IS NULL;
```

Can test for NULL explicitly

attribute IS NULL

attribute IS NOT NULL

sID	sName	GPA
123	Amy	3.9
234	Bob	3.6
345	Craig	3.5
456	Doris	3.9
567	Edward	2.9
678	Fay	3.8
789	Gary	3.4
987	Helen	3.7
876	Irene	3.9
765	Jay	2.9
654	Amy	3.9
543	Craig	3.4
432	Kevin	
321	Lori	

A fifth query with NULL values

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT sID, sName, GPA, HS
FROM Student
WHERE GPA > 3.5 OR HS < 1600;
```

Although Kevin has a NULL GPA, he is retrieved because it has a HS<1600

sID	sName	GPA	HS
123	Amy	3.9	1000
234	Bob	3.6	1500
345	Craig	3.5	500
456	Doris	3.9	1000
678	Fay	3.8	200
789	Gary	3.4	800
987	Helen	3.7	800
876	Irene	3.9	400
765	Jay	2.9	1500
654	Amy	3.9	1000
432	Kevin		1500

NULL values and aggregate functions

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

```
SELECT distinct GPA
FROM Student;
```

GPA
NULL
2.9
3.4
3.5
3.6
3.7
3.8
3.9

} 8 tuples

```
SELECT count(distinct GPA)
FROM Student;
```

count(distinct GPA)
7

When counting (the distinct) values, NULLs are not included

NULL values

During joins, one NULL value is not considered equal to another NULL.

When using a database with NULL values

- Be careful when writing queries

- Understand how the NULL values are going to influence the result

Agenda

Introduction

The JOIN family of operators

Basic SELECT Statement

Aggregation

Table Variables and Set Operators

Null values

Subqueries in WHERE clauses

Data Modification statements

Subqueries in FROM and SELECT clauses

Inserting new data

Insert Into **Table** (**A_1** , **A_2** , ..., **A_n**)

Values (**v_1** , **v_2** , ..., **v_n**)



If we provide values for all attributes, we may omit the list of attributes.

Insert Into **Table**

Select-Statement



Same schema as the table

Inserting new data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Insert Into College

Values ('Carnegie Mellon', 'PA', 11500);

College

cName	state	enr
Stanford	CA	15000
Berkeley	CA	36000
MIT	MA	10000
Cornell	NY	21000
Carnegie Mellon	PA	11500

Inserting new data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Have all students who didn't apply anywhere apply to CS at Carnegie Mellon

```
SELECT *  
FROM Student  
WHERE sID not in (select sID FROM Apply);
```

```
INSERT INTO Apply  
SELECT sID, 'Carnegie Mellon', 'CS', NULL  
FROM Student  
WHERE sID not in (select sID FROM Apply);
```

sID	sName	GPA	HS
456	Doris	3.9	1000
567	Edward	2.9	2000
789	Gary	3.4	800
654	Amy	3.9	1000

Apply

sID	cName	major	dec
...	...	...	...
543	MIT	CS	N
456	Carnegie Mellon	CS	NULL
567	Carnegie Mellon	CS	NULL
789	Carnegie Mellon	CS	NULL
654	Carnegie Mellon	CS	NULL

Inserting new data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Admit to Carnegie Mellon EE all students who were turned down in EE elsewhere

```
SELECT *  
FROM Student  
WHERE sID in (select sID FROM Apply WHERE major='EE' AND  
decision='N' AND cName<>'Carnegie Mellon');
```

sID	sName	GPA	HS
123	Amy	3.9	1000
345	Craig	3.5	500

```
INSERT INTO Apply  
SELECT sID, 'Carnegie Mellon', 'EE', 'Y'  
FROM Student  
WHERE sID in (select sID FROM Apply WHERE major='EE' AND  
decision='N' AND cName<>'Carnegie Mellon');
```

Apply

sID	cName	major	dec
...	...	...	...
654	Carnegie Mellon	CS	NULL
123	Carnegie Mellon	EE	Y
345	Carnegie Mellon	EE	Y

Deleting existing data

Delete From **Table**

Where **Condition**

Condition can be complicated

Can include subqueries and aggregation over other tables

Deleting existing data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Delete all students who applied to more than two different majors

```
SELECT sID
FROM Apply
GROUP BY sID
HAVING count(distinct major) > 2;

DELETE FROM Student
WHERE sID in (
SELECT sID FROM Apply GROUP BY sID
HAVING count(distinct major) > 2);
```

sID

345

876

Student

sID	sName	GPA	HS
123	Amy	3.9	1000
234	Bob	3.6	1500
456	Doris	3.9	1000
567	Edward	2.9	2000
678	Fay	3.8	200
789	Gary	3.4	800
987	Helen	3.7	800
765	Jay	2.9	1500
654	Amy	3.9	1000
543	Craig	3.4	2000

Deleting existing data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Delete those students also from Apply

```
DELETE FROM Apply
```

```
WHERE sID in (
```

```
    SELECT sID FROM Apply GROUP BY sID
```

```
    HAVING count(distinct major) > 2);
```

Not all database systems allow deletion commands where the subquery includes the same relation that you're deleting from

In systems that don't allow, a temporary table would have to be created

Deleting existing data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Delete colleges with no CS applicants

```
SELECT *  
FROM College  
WHERE cName not in  
(SELECT cName from Apply where major='CS');
```

cName	state	enr
Cornell	NY	21000

```
DELETE FROM College  
WHERE cName not in  
(SELECT cName from Apply where major='CS');
```

cName	state	enr
Stanford	CA	15000
Berkeley	CA	36000
MIT	MA	10000
Carnegie Mellon	PA	11500

College

Updating existing data

Update **Table**

Set **Attr** = **Expression**

Where **Condition**

Update **Table**

Set **$A_1 = \text{Expr}_1, A_2 = \text{Expr}_2, \dots, A_n = \text{Expr}_n$**

Where **Condition**

Conditions and expressions can include subqueries and queries over other tables or the same table

Updating existing data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Accept applicants to Carnegie Mellon with GPA<3.6 but turn them into economics majors

```
SELECT *  
FROM Apply  
WHERE cName = 'Carnegie Mellon' and sID in  
(SELECT sID from Student where GPA<3.6);
```

sID	cName	major	dec
567	Carnegie Mellon	CS	
789	Carnegie Mellon	CS	

```
UPDATE Apply  
SET decision = 'Y', major='economics'  
WHERE cName = 'Carnegie Mellon' and sID in  
(SELECT sID from Student where GPA<3.6);
```

Apply

sID	cName	major	dec
...	...	...	...
654	Carnegie Mellon	CS	NULL
123	Carnegie Mellon	EE	Y
567	Carnegie Mellon	economics	Y
789	Carnegie Mellon	economics	Y

Updating existing data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Turn the highest GPA EE applicant into a CSE applicant

```
SELECT *
FROM Apply
WHERE major='EE' AND sID in
(select sID from Student where GPA>=all
    (select GPA from Student where sID in
        (select sID from Apply where major='EE')));
```

sID	cName	major	dec
123	Stanford	EE	N
123	Cornell	EE	Y
123	Carnegie Mellon	EE	Y

Updating existing data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Turn the highest GPA EE applicant into a CSE applicant

UPDATE Apply

SET major = 'CSE'

WHERE major='EE' AND sID in

(select sID from Student where GPA>=all

(select GPA from Student where sID in

(select sID from Apply where major='EE')));

sID	cName	major	dec
...	...	...	...
789	Carnegie Mellon	economics	Y
123	Stanford	CSE	N
123	Cornell	CSE	Y
123	Carnegie Mellon	CSE	Y

Apply

Updating existing data

College(cName, state, enr)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Give every student the highest GPA and the smallest high school in the database

UPDATE Student

SET GPA = (select max(GPA) from Student),
sizeHS = (select min(sizeHS) from Student);

In the SET command, the right-hand side of the equals can itself be a subquery

Student

sID	sName	GPA	HS
123	Amy	3.9	200
234	Bob	3.9	200
456	Doris	3.9	200
567	Edward	3.9	200
678	Fay	3.9	200
789	Gary	3.9	200
987	Helen	3.9	200
765	Jay	3.9	200
654	Amy	3.9	200
543	Craig	3.9	200

Summary

SQL is a rich programming language that handles the way data is processed declaratively

Readings

Jeffrey Ullman, Jennifer Widom, A first course in Database Systems 3rd Edition

- Section 6.1 – Simple Queries in SQL

- Section 6.2 – Queries Involving More Than One Relation

- Section 6.3 - Subqueries

- Section 6.4 – Full-Relation Operations

- Section 6.5 – Database Modifications

Philip Greenspun, SQL for Web Nerds, <http://philip.greenspun.com/sql/>